

To do this, you must modify the lexer and parser specifications in `CLex.fs` and `CPar.fsy`. You may also extend the micro-C abstract syntax in `Absyn.fs` by defining a new statement constructor `Forloop` in the `stmt` type, and add a suitable case to the `exec` function in the interpreter.

But actually, with a modest amount of cleverness (highly recommended), you do not need to introduce special abstract syntax for for-loops, and need not modify the interpreter at all. Namely, a for-loop of the general form

```
for (e1; e2; e3)
  stmt
```

is equivalent to a block

```
{
  e1;
  while (e2) {
    stmt
    e3;
  }
}
```

Hence it suffices to let the semantic action `...` in the parser construct abstract syntax using the existing `Block`, `While`, and `Expr` constructors from the `stmt` type.

Rewrite your programs from Exercise 7.2 to use for-loops instead of while-loops.

Exercise 7.4 Extend the micro-C abstract syntax in `Absyn.fs` with the preincrement and predecrement operators known from C, C++, Java, and C#:

```
type expr =
  ...
  | PreInc of access (* C/C++/Java/C# ++i or ++a[e] *)
  | PreDec of access (* C/C++/Java/C# --i or --a[e] *)
```

Note that the predecrement and preincrement operators work on lvalues, that is, variables and array elements, and more generally on any expression that evaluates to a location.

Modify the micro-C interpreter in `Interp.fs` to handle `PreInc` and `PreDec`. You will need to modify the `eval` function, and use the `getSto` and `setSto` store operations (Sect. 7.3).

Exercise 7.5 Extend the micro-C lexer and parser to accept `++e` and `--e` also, and to build the corresponding abstract syntax.

Exercise 7.6 Add compound assignments `+=` and `*=` and so on to micro-C, that is, lexer, parser, abstract syntax and interpreter (`eval` function). Just as for ordinary assignment, the left-hand side of a compound assignment must be an lvalue, but it is used also as an rvalue.

A beautiful and concise — less than 70 pages — textbook treatment of both (recursive descent) parsing and code generation for a small imperative language was given in 1976 by Niklaus Wirth [11, Chap. 5].

A widely used compiler text is the “dragon book”, named for its cover picture, originally written in 1977 by Alfred Aho and Jeffrey Ullman. The most recent edition from 2006 is by Aho, Lam, Sethi and Ullman [1] and gives a very comprehensive treatment of compilation techniques.

Andrew Appel takes an approach more influenced by functional programming techniques in a series of textbooks [2] using Standard ML, C or Java as implementation language.

Grune has a comprehensive list [6] of compilation-related literature before 1980.

8.11 Exercises

The main goal of these exercises is to familiarize yourself with the compilation of micro-C to bytecode, and the abstract machine used to execute the bytecode.

Exercise 8.1 Download `microc.zip` from the book homepage, unpack it to a folder `MicroC`, and build the micro-C compiler as explained in `README.TXT` step (B).

(i) As a warm-up, compile one of the micro-C examples provided, such as that in source file `ex11.c`, then run it using the abstract machine implemented in Java, as described also in step (B) of the `README` file. When run with command line argument 8, the program prints the 92 solutions to the eight queens problem: how to place eight queens on a chessboard so that none of them can attack any of the others.

(ii) Now compile the example micro-C programs `ex3.c` and `ex5.c` using functions `compileToFile` and `fromFile` from `ParseAndComp.fs` as above.

Study the generated symbolic bytecode. Write up the bytecode in a more structured way with labels only at the beginning of the line (as in this chapter). Write the corresponding micro-C code to the right of the stack machine code. Note that `ex5.c` has a nested scope (a block `...` inside a function body); how is that visible in the generated code?

Execute the compiled programs using `java Machine ex3.out 10` and similar. Note that these micro-C programs require a command line argument (an integer) when they are executed.

Trace the execution using `java Machinetrace ex3.out 4`, and explain the stack contents and what goes on in each step of execution, and especially how the low-level bytecode instructions map to the higher-level features of `MicroC`. You can capture the standard output from a command prompt (in a file `ex3trace.txt`) using the Unix-style notation:

```
java Machinetrace ex3.out 4 > ex3trace.txt
```

Exercise 8.2 Compile and run the micro-C example programs you wrote in Exercise 7.2, and check that they produce the right result. It is rather cumbersome to fill an array with values by hand in micro-C, so the function `squares` from that exercise is very handy.

Exercise 8.3 This abstract syntax for preincrement `++e` and predecrement `--e` was introduced in Exercise 7.4:

```
type expr =
  ...
  | PreInc of access (* C/C++/Java/C# ++i or ++a[e] *)
  | PreDec of access (* C/C++/Java/C# --i or --a[e] *)
```

Modify the compiler (function `cExpr`) to generate code for `PreInc(acc)` and `PreDec(acc)`. To parse micro-C source programs containing these expressions, you also need to modify the lexer and parser.

It is tempting to expand `++e` to the assignment expression `e = e+1`, but that would evaluate `e` twice, which is wrong. Namely, `e` may itself have a side effect, as in `++arr[++i]`.

Hence `e` should be computed only once. For instance, `++i` should compile to something like this: `<code to compute address of i>, DUP, LDI, CSTI 1, ADD, STI`, where the address of `i` is computed once and then duplicated.

Write a program to check that this works. If you are brave, try it on expressions of the form `++arr[++i]` and check that `i` and the elements of `arr` have the correct values afterwards.

Exercise 8.4 Compile `ex8.c` and study the symbolic bytecode to see why it is so much slower than the handwritten 20 million iterations loop in `prog1`.

Compile `ex13.c` and study the symbolic bytecode to see how loops and conditionals interact; describe what you see.

In a later chapter we shall see an improved micro-C compiler that generates fewer extraneous labels and jumps.

Exercise 8.5 Extend the micro-C language, the abstract syntax, the lexer, the parser, and the compiler to implement conditional expressions of the form `(e1 ? e2 : e3)`.

The compilation of `e1 ? e2 : e3` should produce code that evaluates `e2` only if `e1` is true and evaluates `e3` only if `e1` is false. The compilation scheme should be the same as for the conditional statement `if (e) stmt1 else stmt2`, but expression `e2` or expression `e3` must leave its value on the stack top if evaluated, so that the entire expression `e1 ? e2 : e3` leaves its value on the stack top.